

Mastering Relational and Non-Relational Database

Paulo André Carminati – RM557881 2TDSPZ

Arthur Bispo de Lima – RM 557568 2TDSPV

João Paulo Moreira - RM557808 2TDSPV

Link do vídeo aplicação java: [youtube&logoColor=white](https://www.youtube.com/watch?v=...&logoColor=white) alt="Vídeo de Apresentação" data-bbox="124 266 725 279"/>

Bem-vindo ao GS Alerta Desastres! Uma aplicação full-stack robusta e interativa, projetada para monitorar eventos de desastres naturais em tempo real, fornecer informações cruciais e permitir o disparo de alertas para usuários cadastrados. Este projeto foi desenvolvido com dedicação pela equipe **MetaMind** para a Global Solution 2025 (1º Semestre) da FIAP.

Índice

- [Mastering Relational and Non-Relational Database](#)
 - [Índice](#)
 - [1. Visão Geral](#) 
 - [2. Estrutura do Banco de Dados](#) 
 - [2.1. Tabelas Principais](#)
 - [2.2. Tabelas de Junção \(Relacionamento\)](#)
 - [2.3. Sequences](#)
 - [2.4. Triggers](#)
 - [2.5. Constraints e Chaves](#)
 - [3. Funcionalidades Implementadas \(PL/SQL\)](#) 
 - [3.1. Procedures de Inserção, Atualização e Exclusão](#)
 - [3.2. Funções de Processamento de Dados](#)
 - [4. Consultas Complexas \(Relatórios SQL\)](#) 
 - [5. Blocos Anônimos e Cursores](#) 
 - [6. Link do Repositório](#)
 - [7. Tecnologias Utilizadas](#)
-

1. Visão Geral

O banco de dados foi desenvolvido para centralizar e gerenciar informações cruciais sobre clientes, incluindo seus dados pessoais, múltiplos contatos e endereços. O sistema integra dados de eventos naturais provenientes do **EONET (Earth Observatory Natural Event Tracker)**, permitindo associar esses eventos a localizações geográficas e analisar o impacto em relação aos clientes. A arquitetura é normalizada, garantindo consistência e minimizando a redundância de dados.

2. Estrutura do Banco de Dados

O esquema é composto por tabelas principais para armazenar entidades distintas, tabelas de junção para gerenciar relacionamentos muitos-para-muitos, sequences para geração automática de IDs e triggers para automatizar a inserção de chaves primárias.

2.1. Tabelas Principais

- **TB_CLIENTE3** : Armazena os dados cadastrais dos clientes.

Colunas: ID_CLIENTE (PK), NOME , SOBRENOME , DOCUMENTO , DATA_NASCIMENTO .

- **TB_CONTATO3** : Mantém os diversos tipos de contato associados aos clientes.

Colunas: ID_CONTATO (PK), DDD , TELEFONE , CELULAR , WHATSAPP , EMAIL ,

TIPO_CONTATO .

- **TB_ENDERECO3** : Contém as informações detalhadas dos endereços.

Colunas: ID_ENDERECO (PK), CEP , LOGRADOURO , NUMERO , BAIRRO , LOCALIDADE , UF ,

COMPLEMENTO , LATITUDE , LONGITUDE .

- **TB_EONET3** : Armazena dados sobre eventos naturais obtidos do EONET.

Colunas: ID_EONET (PK), EONET_ID (ID externo), DATA , JSON (detalhes do evento).

2.2. Tabelas de Junção (Relacionamento)

- **TB_CLIENTECONTATO3** : Associa clientes a seus múltiplos contatos (N:M).
- **TB_CLIENTEENDERECO3** : Associa clientes a seus múltiplos endereços (N:M).
- **TB_ENDERECOEVENTOS3** : Associa endereços a eventos EONET (N:M).

2.3. Sequences

Quatro sequences são utilizadas para gerar automaticamente os valores das chaves primárias, garantindo IDs únicos: -

TB_CLIENTE3_ID_CLIENTE_SEQ - TB_CONTATO3_ID_CONTATO_SEQ - TB_ENDERECO3_ID_ENDERECO_SEQ -

TB_EONET3_ID_EONET_SEQ

2.4. Triggers

⚡ Para cada tabela principal, um trigger BEFORE INSERT utiliza a sequence correspondente para popular a coluna de ID automaticamente. - TB_CLIENTE3_ID_CLIENTE_TRG -

TB_CONTATO3_ID_CONTATO_TRG

-

TB_ENDERECO3_ID_ENDERECO_TRG

-

TB_EONET3_ID_EONET_TRG

2.5. Constraints e Chaves

- **Chaves Primárias (PK):** Cada tabela possui uma chave primária para garantir a unicidade.

- **Chaves Estrangeiras (FK):** Definem e mantêm a integridade referencial entre as tabelas.
- **NOT NULL :** Colunas essenciais são definidas como NOT NULL para garantir que sempre contenham dados.

3. Funcionalidades Implementadas (PL/SQL) ⚙

Para cada tabela, foram criadas procedures e funções para encapsular as operações de DML, promovendo reusabilidade e segurança.

3.1. Procedures de Inserção, Atualização e Exclusão

Uma API completa de procedures foi criada para gerenciar o ciclo de vida dos dados:

- **P_..._INSERT** : Adicionam novos registros. Ex: P_CLIENTE3_INSERT .
- **P_..._UPDATE** : Modificam dados de registros existentes. Ex: P_CLIENTE3_UPDATE .
- **P_..._DELETE** : Removem registros e suas associações para manter a integridade. Ex: P_CLIENTE3_DELETE .

<details> <summary> 📄 Exemplo de Procedure de Inserção</summary>

```
-- Insere um novo cliente e retorna o ID gerado.
CREATE OR REPLACE PROCEDURE P_CLIENTE3_INSERT (
    p_nome          IN TB_CLIENTE3.NOME%TYPE,
    p_sobrenome     IN TB_CLIENTE3.SOBRENOME%TYPE,
    p_documento     IN TB_CLIENTE3.DOCUMENTO%TYPE,
    p_data_nascimento IN TB_CLIENTE3.DATA_NASCIMENTO%TYPE,
    p_new_id        OUT TB_CLIENTE3.ID_CLIENTE%TYPE
) IS
BEGIN
    INSERT INTO TB_CLIENTE3 (NOME, SOBRENOME, DOCUMENTO, DATA_NASCIMENTO)
    VALUES (p_nome, p_sobrenome, p_documento, p_data_nascimento)
    RETURNING ID_CLIENTE INTO p_new_id;
END P_CLIENTE3_INSERT;
/
```

</details>

3.2. Funções de Processamento de Dados

Funções foram desenvolvidas para encapsular cálculos e lógicas de negócio.

- **F_TOTAL_CLIENTES_POR_UF(p_uf VARCHAR2) RETURN NUMBER**
Conta o número de clientes únicos que possuem endereço em um determinado estado (UF).
- **F_CALCULAR_IDADE_CLIENTE(p_id_cliente NUMBER) RETURN NUMBER**
Calcula a idade de um cliente com base em sua data de nascimento (armazenada como VARCHAR2).

<details> <summary> 📄 Exemplo de Função</summary>

```

-- Calcula a idade de um cliente.
CREATE OR REPLACE FUNCTION F_CALCULAR_IDADE_CLIENTE (

    p_id_cliente IN NUMBER

) RETURN NUMBER IS
    v_data_nasc_varchar VARCHAR2(10);
    v_data_nasc_date     DATE;
    v_idade              NUMBER;
BEGIN
    SELECT DATA_NASCIMENTO INTO v_data_nasc_varchar
    FROM TB_CLIENTE3 WHERE ID_CLIENTE = p_id_cliente;

    v_data_nasc_date := TO_DATE(v_data_nasc_varchar, 'DD/MM/YYYY');
    v_idade := TRUNC(MONTHS_BETWEEN(SYSDATE, v_data_nasc_date) / 12);

    RETURN v_idade;
EXCEPTION
    WHEN OTHERS THEN
        RETURN NULL;
END F_CALCULAR_IDADE_CLIENTE;
/

```

</details>

4. Consultas Complexas (Relatórios SQL)

Foram desenvolvidas 5 consultas complexas para extrair informações gerenciais, utilizando JOIN, GROUP BY, HAVING, subqueries e funções de agregação:

- 🗺️ **Relatório de Clientes por Estado (UF):** Ranking de estados por número de clientes.
- **Relatório de Contatos por Cliente:** Quantidade de contatos registrados para cada cliente.
- **Relatório de Eventos de Alto Impacto:** Identifica eventos EONET que afetaram múltiplos endereços.
- **Relatório de Perfil Detalhado do Cliente:** Usa subqueries para exibir um perfil consolidado.
- 🏠 **Análise de Média de Clientes por Cidade em SP:** Calcula a média de clientes por cidade em um estado específico.

<details> <summary>  Exemplo de Relatório (Eventos de Alto Impacto) </summary>

```

-- Identifica eventos que afetaram um ou mais endereços cadastrados.
SELECT
    evt.EONET_ID,
    -- Aplicamos MAX() ao título. Como só há um título por evento, o
    resultado é o mesmo.
    MAX(JSON_VALUE(evt.JSON, '$.title')) AS titulo_do_evento,
    COUNT(ee.TB_ENDERECO3_ID_ENDERECO) AS enderecos_afetados
FROM
    TB_EONET3 evt
JOIN
    TB_ENDERECOEVENTOS3 ee ON evt.ID_EONET = ee.TB_EONET3_ID_EONET
GROUP BY

    -- Agrupamos pelo identificador único do evento para corrigir o erro
    ORA-00979.
    evt.ID_EONET,
    evt.EONET_ID
HAVING
    COUNT(ee.TB_ENDERECO3_ID_ENDERECO) > 0
ORDER BY
    enderecos_afetados DESC;

```

</details>

5. Blocos Anônimos e Cursores

Blocos PL/SQL foram criados para demonstrar lógicas de negócio e o uso de cursores.

5.1. Lógica de Negócio com Blocos Anônimos

- **Análise Regional:** Um bloco que gera um relatório de clientes por estado, usando LOOP e IF/ ELSE para destacar regiões com alta concentração.
- **Análise de Risco:** Um bloco que percorre clientes e verifica a associação com eventos naturais, emitindo alertas.

5.2. Demonstração de Cursor Explícito

Foi criado um bloco anônimo para demonstrar o controle manual de um cursor com OPEN, FETCH e CLOSE para processar dados linha a linha, gerando um relatório de clientes.

<details> <summary>📄 Exemplo de Cursor Explícito</summary>

```

DECLARE
    -- Declaração das variáveis que receberão os dados
    v_nome_completo VARCHAR2(201);
    v_documento      TB_CLIENTE3.DOCUMENTO%TYPE;

    -- Declaração do CURSOR EXPLÍCITO
    CURSOR c_lista_clientes IS
        SELECT NOME || ' ' || SOBRENOME, DOCUMENTO
        FROM TB_CLIENTE3 ORDER BY NOME;
BEGIN
    -- 1. OPEN: Abre o cursor
    OPEN c_lista_clientes;
    -- 2. LOOP e FETCH: Busca os dados
    LOOP
        FETCH c_lista_clientes INTO v_nome_completo, v_documento;
        EXIT WHEN c_lista_clientes%NOTFOUND;
        -- Processa os dados
        DBMS_OUTPUT.PUT_LINE(RPAD(v_nome_completo, 40) || v_documento);
    END LOOP;

    -- 3. CLOSE: Fecha o cursor
    CLOSE c_lista_clientes;
END;
/

```

</details>

6. Link do Repositório:

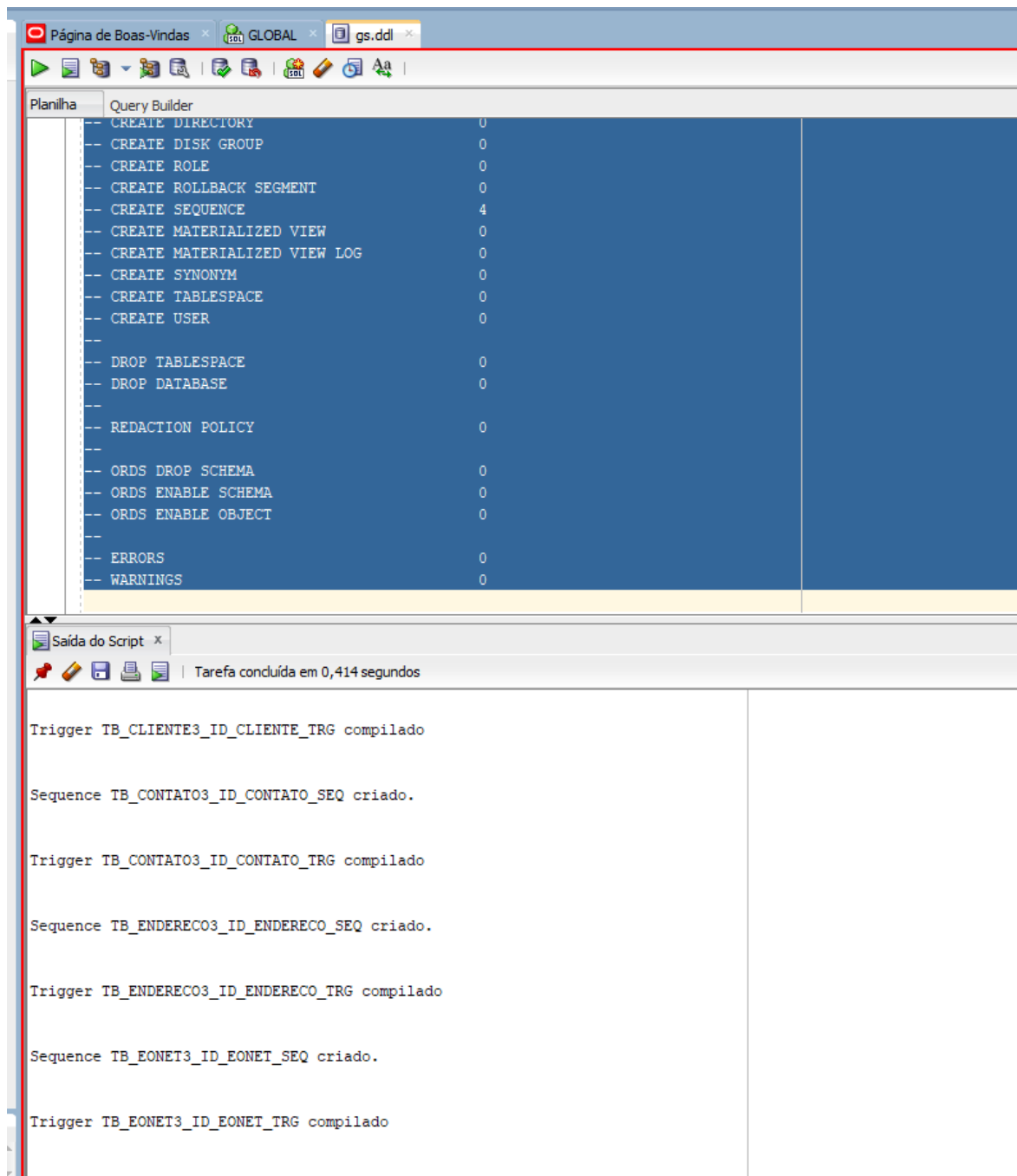
[GitHub](#)

7. Tecnologias Utilizadas:

Oracle Database

PL/SQL

Imagens da execução do arquivo sql:



The screenshot displays the SQL Developer interface during the execution of a script named 'gs.ddl'. The top pane, titled 'Planilha', shows the script content with counts for various SQL statements. The bottom pane, titled 'Saída do Script', shows the output of the script execution.

Script Content (Top Pane):

Statement	Count
CREATE DIRECTORY	0
CREATE DISK GROUP	0
CREATE ROLE	0
CREATE ROLLBACK SEGMENT	0
CREATE SEQUENCE	4
CREATE MATERIALIZED VIEW	0
CREATE MATERIALIZED VIEW LOG	0
CREATE SYNONYM	0
CREATE TABLESPACE	0
CREATE USER	0
DROP TABLESPACE	0
DROP DATABASE	0
REDACTION POLICY	0
ORDS DROP SCHEMA	0
ORDS ENABLE SCHEMA	0
ORDS ENABLE OBJECT	0
ERRORS	0
WARNINGS	0

Script Output (Bottom Pane):

```
Trigger TB_CLIENTE3_ID_CLIENTE_TRG compilado

Sequence TB_CONTATO3_ID_CONTATO_SEQ criado.

Trigger TB_CONTATO3_ID_CONTATO_TRG compilado

Sequence TB_ENDERECO3_ID_ENDERECO_SEQ criado.

Trigger TB_ENDERECO3_ID_ENDERECO_TRG compilado

Sequence TB_EONET3_ID_EONET_SEQ criado.

Trigger TB_EONET3_ID_EONET_TRG compilado
```

Oracle SQL Developer

Arquivo Editar Exibir Navegar Executar Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

GLOBAL

Tabelas (Filtrado)

TB_CLIENTE3

TB_CLIENTECONTATO3

TB_CLIENTEENDereco3

TB_CONTATO3

TB_ENDereco3

TB_ENDerecoCOEVENOS3

TB_FONET3

Views

Índices

Pacotes

Procedures

P_CLIENTE3_DELETE

P_CLIENTE3_INSERT

P_CLIENTE3_UPDATE

P_CLIENTECONTATO3_INSERT

P_CLIENTEENDereco3_INSERT

P_CONTATO3_DELETE

P_CONTATO3_INSERT

P_CONTATO3_UPDATE

P_ENDereco3_DELETE

P_ENDereco3_INSERT

P_ENDereco3_UPDATE

P_ENDerecoCOEVENOS3_INSERT

P_FONET3_INSERT

Funções

Operadores

Flas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinônimos

Sinônimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Diretórios

Edições

Esquemas XML

Java

Repositório de BD XML

Opção OLAP

Relatórios

Todos os Relatórios

Relatórios Definidos pelo Usuário

Planilha Query Builder

```

P_CLIENTEENDereco3_INSERT(v_id_cliente2, v_id_endereco2);
P_CLIENTECONTATO3_INSERT(v_id_cliente2, v_id_contato2);
P_CLIENTEENDereco3_INSERT(v_id_cliente2, v_id_endereco2);

P_CLIENTECONTATO3_INSERT(v_id_cliente3, v_id_contato3);
P_CLIENTEENDereco3_INSERT(v_id_cliente3, v_id_endereco3);

P_CLIENTECONTATO3_INSERT(v_id_cliente4, v_id_contato4);
P_CLIENTEENDereco3_INSERT(v_id_cliente4, v_id_endereco4);

P_CLIENTECONTATO3_INSERT(v_id_cliente5, v_id_contato5);
P_CLIENTEENDereco3_INSERT(v_id_cliente5, v_id_endereco5);

-- == DEMONSTRANDO UPDATE E DELETE ==
DBMS_OUTPUT.PUT_LINE('-- Demonstrando UPDATE e DELETE --');

-- Atualizando o sobrenome do cliente 5
P_CLIENTE3_UPDATE(v_id_cliente5, 'Ricardo', 'Gomes da Silva', '456.789.012-13', '30/07/1975');

-- Deletando o cliente 3 (Beatriz) e todas as suas associações
P_CLIENTE3_DELETE(v_id_cliente3);

```

Salida do Script

Tarefa concluída em 0,395 segundos

Endereço "Avenida Brigadeiro Faria Lima" inserido. Novo ID: 4
Endereço "Rua 24 Horas" inserido. Novo ID: 5
-- Associando Clientes com Contatos e Endereços --
Associado cliente 1 com contato 1
Associado cliente 1 com endereço 1
Associado cliente 2 com contato 2
Associado cliente 2 com endereço 2
Associado cliente 3 com contato 3
Associado cliente 3 com endereço 3
Associado cliente 4 com contato 4
Associado cliente 4 com endereço 4
Associado cliente 5 com contato 5
Associado cliente 5 com endereço 5
-- Demonstrando UPDATE e DELETE --
Cliente ID 5 atualizado com sucesso.
Cliente ID 3 e suas associações foram excluídos.
----> SUCESSO! Dados inseridos e COMMIT realizado. <----

Procedimento PL/SQL concluído com sucesso.

Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

GLOBAL

Tabelas (Filtrado)

TB_CLIENTE3

TB_CLIENTECONTATO3

TB_CLIENTEENDereco3

TB_CONTATO3

TB_ENDereco3

TB_ENDerecoCOEVENOS3

TB_FONET3

Views

Índices

Pacotes

Procedures

P_CLIENTE3_DELETE

P_CLIENTE3_INSERT

P_CLIENTE3_UPDATE

P_CLIENTECONTATO3_INSERT

P_CLIENTEENDereco3_INSERT

P_CONTATO3_DELETE

P_CONTATO3_INSERT

P_CONTATO3_UPDATE

P_ENDereco3_DELETE

P_ENDereco3_INSERT

P_ENDereco3_UPDATE

P_ENDerecoCOEVENOS3_INSERT

P_FONET3_INSERT

Funções

Operadores

Flas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinônimos

Sinônimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Planilha Query Builder

```

SELECT
  'São Paulo (SP)' AS ESTADO,
  F_TOTAL_CLIENTES_POR_UF('SP') AS TOTAL_CLIENTES
FROM DUAL
UNION ALL
SELECT
  'Rio de Janeiro (RJ)' AS ESTADO,
  F_TOTAL_CLIENTES_POR_UF('RJ') AS TOTAL_CLIENTES
FROM DUAL;

```

Salida do Script

Resultado da Consulta

Todas as Linhas Extraídas: 2 em 0,036 segundos

ESTADO	TOTAL_CLIENTES
1 São Paulo (SP)	2
2 Rio de Janeiro (RJ)	1

Oracle SQL Developer

Arquivo Editar Exibir Navegar Executar Equipe Ferramentas Janela Ajuda

Conexões GLOBAL

Planilha Query Builder

```

CREATE OR REPLACE FUNCTION F_TOTAL_CLIENTES_POR_UF (
  p_uf IN VARCHAR2
) RETURN NUMBER IS
  v_total_clientes NUMBER;
BEGIN
  -- Conta os IDs de clientes distintos que possuem endereço no UF especificado
  SELECT COUNT(DISTINCT cli.ID_CLIENTE)
  INTO v_total_clientes
  FROM TB_CLIENTE3 cli
  JOIN TB_CLIENTEENDereco3 ce ON cli.ID_CLIENTE = ce.TB_CLIENTE3_ID_CLIENTE
  JOIN TB_ENDereco3 ender ON ce.TB_ENDereco3_ID_ENDereco = ender.ID_ENDereco
  WHERE ender.UF = UPPER(p_uf); -- Garante que a comparação não seja sensível a maiúsculas/minúsculas

  RETURN v_total_clientes;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 0;
  WHEN OTHERS THEN
    RETURN -1; -- Retorna -1 em caso de outro erro
END F_TOTAL_CLIENTES_POR_UF;

```

Saída do Script x

Tarefa concluída em 0,048 segundos

Function F_TOTAL_CLIENTES_POR_UF compilado

Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões GLOBAL

Planilha Query Builder

```

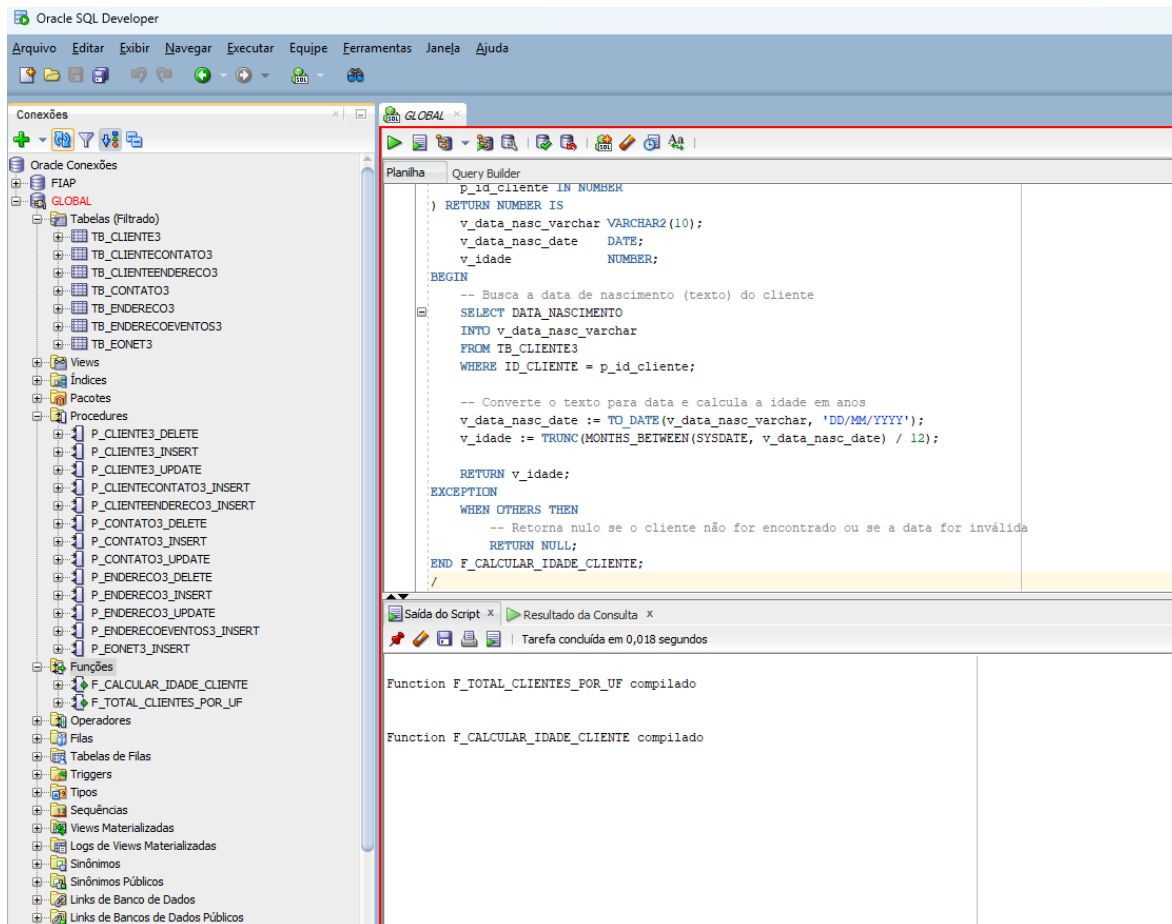
SELECT
  ID_CLIENTE,
  NOME || ' ' || SOBRENOME AS NOME_COMPLETO,
  DATA_NASCIMENTO,
  F_CALCULAR_IDADE_CLIENTE(ID_CLIENTE) AS IDADE_CALCULADA
FROM
  TB_CLIENTE3;

```

Saída do Script x Resultado da Consulta x

Todas as Linhas Extraídas: 4 em 0,014 segundos

ID_CLIENTE	NOME_COMPLETO	DATA_NASCIMENTO	IDADE_CALCULADA
1	1 Mariana Costa	10/05/1990	35
2	2 Fernando Almeida	22/11/1982	42
3	4 GlobalTech S.A.	01/01/2015	10
4	5 Ricardo Gomes da Silva	30/07/1975	49



Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDERECO3
- TB_CONTATO3
- TB_ENDERECO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedimentos

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDERECO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDERECO3_DELETE
- P_ENDERECO3_INSERT
- P_ENDERECO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Filas

Tabelas de Filas

Triggers

Views Materializadas

Logs de Views Materializadas

Sinônimos

Sinônimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Diretórios

Edições

Esquemas XML

Java

Relatórios

Todos os Relatórios

Relatórios Definidos pelo Usuário

Oracle SQL Developer : GLOBAL

Planilha Query Builder

```

BEGIN
    DBMS_OUTPUT.PUT_LINE('--- RELATÓRIO DE CONCENTRAÇÃO DE CLIENTES POR ESTADO ---');
    DBMS_OUTPUT.PUT_LINE(RPAD('UF', 5) || RPAD('TOTAL CLIENTES', 20) || 'EXEMPLO DE CLIENTE');
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 4, '-') || ' ' || RPAD('-', 19, '-') || ' ' || RPAD('-', 30, '-'));

    -- LOOP para percorrer cada registro retornado pelo cursor
    FOR reg IN c_clientes_por_uf LOOP
        DBMS_OUTPUT.PUT(RPAD(reg.UF, 5));
        DBMS_OUTPUT.PUT(RPAD(reg.total_clientes, 20));
        DBMS_OUTPUT.PUT_LINE(reg.exemplo_cliente);

        -- Estrutura IF/ELSE para adicionar uma observação
        IF reg.total_clientes >= v_limite_alerta THEN
            DBMS_OUTPUT.PUT_LINE('    -> OBS: Alta concentração de clientes neste estado!');
        ELSE
            DBMS_OUTPUT.PUT_LINE('    -> OBS: Concentração normal. ');
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('--- FIM DO RELATÓRIO ---');
END;
/

```

Salida do Script x Resultado da Consulta x

Tarefa concluída em 0,077 segundos

Function F_TOTAL_CLIENTES_POR_UF compilado

Function F_CALCULAR_IDADE_CLIENTE compilado

```

--- RELATÓRIO DE CONCENTRAÇÃO DE CLIENTES POR ESTADO ---
UF  TOTAL CLIENTES      EXEMPLO DE CLIENTE
-----
SP  2              Mariana Costa
    -> OBS: Alta concentração de clientes neste estado!
PR  1              Ricardo Gomes da Silva
    -> OBS: Concentração normal.
RJ  1              Fernando Almeida
    -> OBS: Concentração normal.
--- FIM DO RELATÓRIO ---

```

Procedimento PL/SQL concluído com sucesso.

Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDERECO3
- TB_CONTATO3
- TB_ENDERECO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedimentos

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDERECO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDERECO3_DELETE
- P_ENDERECO3_INSERT
- P_ENDERECO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Filas

Tabelas de Filas

Triggers

Views Materializadas

Logs de Views Materializadas

Sinônimos

Sinônimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Diretórios

Edições

Planilha Query Builder

```

-- Para cada cliente, executa uma consulta para contar os eventos associados
SELECT COUNT(evt.ID_EONET)
INTO v_total_eventos
FROM TB_CLIENTEENDERECO3 ee
JOIN TB_ENDERECOEVENTOS3 ee ON ee.TB_ENDERECO3_ID_ENDERECO = ee.TB_ENDERECO3_ID_ENDERECO
JOIN TB_EONET3 evt ON ee.TB_EONET3_ID_EONET = evt.ID_EONET
WHERE ee.TB_CLIENTES_ID_CLIENTE = cliente.ID_CLIENTE;

-- Estrutura IF/ELSE para decidir qual mensagem exibir
IF v_total_eventos > 0 THEN
    DBMS_OUTPUT.PUT_LINE('ALERTA DE RISCO: O cliente ' || cliente.NOME || ' ' || cliente.SOBRENOME
    || ' (ID: ' || cliente.ID_CLIENTE || ') está associado a ' || v_total_eventos || ' even');
ELSE
    DBMS_OUTPUT.PUT_LINE('OK: O cliente ' || cliente.NOME || ' ' || cliente.SOBRENOME
    || ' (ID: ' || cliente.ID_CLIENTE || ') não possui riscos associados. ');
END IF;
END LOOP;

DBMS_OUTPUT.PUT_LINE('--- FIM DA VERIFICAÇÃO ---');
END;
/

```

Salida do Script x Resultado da Consulta x

Tarefa concluída em 0,017 segundos

```

--- RELATÓRIO DE RISCO DE CLIENTES (EVENTOS EONET) ---
OK: O cliente Fernando Almeida (ID: 2) não possui riscos associados.
OK: O cliente GlobalTech S.A. (ID: 4) não possui riscos associados.
OK: O cliente Mariana Costa (ID: 1) não possui riscos associados.
OK: O cliente Ricardo Gomes da Silva (ID: 5) não possui riscos associados.
--- FIM DA VERIFICAÇÃO ---

```

Procedimento PL/SQL concluído com sucesso.

Oracle SQL Developer: GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

FIAP

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDEREÇO3
- TB_CONTATO3
- TB_ENDEREÇO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedures

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDEREÇO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDEREÇO3_DELETE
- P_ENDEREÇO3_INSERT
- P_ENDEREÇO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Flas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinônimos

Sinônimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Diretórios

Edições

Esquemas XML

Java

Planilha Query Builder

```
-- 4. LOOP: Inicia um laço para percorrer os resultados.
LOOP
-- 5. FETCH: Busca a próxima linha do resultado e a carrega nas variáveis.
FETCH c_lista_clientes INTO v_nome_completo, v_documento;

-- 6. EXIT WHEN: Condição de saída do loop.
-- A variável %NOTFOUND do cursor se torna TRUE quando o FETCH não encontra mais linhas.
EXIT WHEN c_lista_clientes%NOTFOUND;

-- Processa os dados da linha atual
DBMS_OUTPUT.PUT_LINE(RPAD(v_nome_completo, 40) || v_documento);

END LOOP;

-- 7. CLOSE: Fecha o cursor, liberando os recursos utilizados por ele.
-- É uma etapa crucial para a boa gestão de memória.
CLOSE c_lista_clientes;

DBMS_OUTPUT.PUT_LINE('--- FIM DO RELATÓRIO ---');

END;
```

Saída do Script x Resultado da Consulta x

Tarefa concluída em 0,017 segundos

--- RELATÓRIO DE CLIENTES (CURSOR EXPLÍCITO) ---

NOME COMPLETO	DOCUMENTO
Fernando Almeida	234.567.890-11
GlobalTech S.A.	01.234.567/0001-88
Mariana Costa	123.456.789-10
Ricardo Gomes da Silva	456.789.012-13

--- FIM DO RELATÓRIO ---

Procedimento PL/SQL concluído com sucesso.

Oracle SQL Developer: GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

FIAP

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDEREÇO3
- TB_CONTATO3
- TB_ENDEREÇO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedures

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDEREÇO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDEREÇO3_DELETE
- P_ENDEREÇO3_INSERT
- P_ENDEREÇO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Flas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Planilha Query Builder

```
SELECT
    ender.UF,
    COUNT(DISTINCT cli.ID_CLIENTE) AS total_de_clientes
FROM
    TB_CLIENTE3 cli
JOIN
    TB_CLIENTEENDEREÇO3 ce ON cli.ID_CLIENTE = ce.TB_CLIENTE3_ID_CLIENTE
JOIN
    TB_ENDEREÇO3 ender ON ce.TB_ENDEREÇO3_ID_ENDEREÇO = ender.ID_ENDEREÇO
GROUP BY
    ender.UF
ORDER BY
    total_de_clientes DESC;
```

Saída do Script x Resultado da Consulta x

Todas as Linhas Extraídas: 3 em 0,003 segundos

UF	TOTAL_DE_CLIENTES
1 SP	2
2 PR	1
3 RJ	1

Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

FIAP

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDERECO3
- TB_CONTATO3
- TB_ENDERECO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedures

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDERECO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDERECO3_DELETE
- P_ENDERECO3_INSERT
- P_ENDERECO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Flas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinônimos

Sinônimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Diretórios

Edições

Esquemas XML

Planilha Query Builder

```

SELECT
  c.NOME || ' ' || c.SOBRENOME AS nome_cliente,
  c.DOCUMENTO,
  COUNT(cc.TB_CONTATO3_ID_CONTATO) AS quantidade_de_contatos
FROM
  TB_CLIENTE3 c
LEFT JOIN -- Usamos LEFT JOIN para listar até mesmo clientes sem contatos
  TB_CLIENTECONTATO3 cc ON c.ID_CLIENTE = cc.TB_CLIENTE3_ID_CLIENTE
GROUP BY
  c.ID_CLIENTE,
  c.NOME,
  c.SOBRENOME,
  c.DOCUMENTO
ORDER BY
  quantidade_de_contatos DESC,
  nome_cliente;

```

Salida do Script x Resultado da Consulta x

Todas as Linhas Extraídas: 4 em 0,045 segundos

NOME_CLIENTE	DOCUMENTO	QUANTIDADE_DE_CONTATOS
1 Fernando Almeida	234.567.890-11	1
2 GlobalTech S.A.	01.234.567/0001-88	1
3 Mariana Costa	123.456.789-10	1
4 Ricardo Gomes da Silva	456.789.012-13	1

Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

FIAP

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDERECO3
- TB_CONTATO3
- TB_ENDERECO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedures

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDERECO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDERECO3_DELETE
- P_ENDERECO3_INSERT
- P_ENDERECO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Filas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinónimos

Sinónimos Públicos

Links de Banco de Dados

Links de Bancos de Dados Públicos

Diretórios

Edições

Esquemas XML

Java

Planilha Query Builder

```

SELECT
  c.NOME,
  c.SOBRENOME,
  -- Subquery para contar o número de endereços
  (SELECT COUNT(*)
   FROM TB_CLIENTEENDERECO3 ce
   WHERE ce.TB_CLIENTE3_ID_CLIENTE = c.ID_CLIENTE) AS total_enderecos,
  -- Subquery para buscar o e-mail principal (o primeiro encontrado)
  (SELECT ct.EMAIL
   FROM TB_CONTATO3 ct
   JOIN TB_CLIENTECONTATO3 cc ON ct.ID_CONTATO = cc.TB_CONTATO3_ID_CONTATO
   WHERE cc.TB_CLIENTE3_ID_CLIENTE = c.ID_CLIENTE AND ROWNUM = 1) AS email_principal
FROM
  TB_CLIENTE3 c
ORDER BY
  c.NOME;

```

Salida do Script Resultado da Consulta

Todas as Linhas Extraídas: 4 em 0,007 segundos

NOME	SOBRENOME	TOTAL_ENDERECOS	EMAIL_PRINCIPAL
1	Fernando	Almeida	1fernando.a@email.com
2	GlobalTech	S.A.	1contato@globaltech.com
3	Mariana	Costa	1mariana.costa@email.com
4	Ricardo	Gomes da Silva	1ricardo.gomes@email.com

Oracle SQL Developer : GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

FIAP

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDERECO3
- TB_CONTATO3
- TB_ENDERECO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedures

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDERECO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDERECO3_DELETE
- P_ENDERECO3_INSERT
- P_ENDERECO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Filas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinónimos

Sinónimos Públicos

Links de Banco de Dados

Planilha Query Builder

```

SELECT
  AVG(total_clientes_por_cidade) AS media_de_clientes_por_cidade_em_sp
FROM
  -- Subquery (ou Visão Inline) que primeiro calcula o total de clientes para cada cidade em SP
  (SELECT
    ender.LOCALIDADE,
    COUNT(DISTINCT ce.TB_CLIENTE3_ID_CLIENTE) AS total_clientes_por_cidade
   FROM
     TB_CLIENTEENDERECO3 ce
   JOIN
     TB_ENDERECO3 ender ON ce.TB_ENDERECO3_ID_ENDERECO = ender.ID_ENDERECO
   WHERE
     ender.UF = 'SP'
   GROUP BY
     ender.LOCALIDADE
  ) relatorio_cidades;

```

Salida do Script Resultado da Consulta

Todas as Linhas Extraídas: 1 em 0,004 segundos

MEDIA_DE_CLIENTES_POR_CIDADE_EM_SP
1

Oracle SQL Developer: GLOBAL

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda

Conexões

Oracle Conexões

FIAP

GLOBAL

Tabelas (Filtrado)

- TB_CLIENTE3
- TB_CLIENTECONTATO3
- TB_CLIENTEENDEREÇO3
- TB_CONTATO3
- TB_ENDEREÇO3
- TB_ENDERECOEVENTOS3
- TB_EONET3

Views

Índices

Pacotes

Procedures

- P_CLIENTE3_DELETE
- P_CLIENTE3_INSERT
- P_CLIENTE3_UPDATE
- P_CLIENTECONTATO3_INSERT
- P_CLIENTEENDEREÇO3_INSERT
- P_CONTATO3_DELETE
- P_CONTATO3_INSERT
- P_CONTATO3_UPDATE
- P_ENDEREÇO3_DELETE
- P_ENDEREÇO3_INSERT
- P_ENDEREÇO3_UPDATE
- P_ENDERECOEVENTOS3_INSERT
- P_EONET3_INSERT

Funções

- F_CALCULAR_IDADE_CLIENTE
- F_TOTAL_CLIENTES_POR_UF

Operadores

Filas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Logs de Views Materializadas

Sinônimos

- Sinônimos Públicos
- Links de Banco de Dados
- Links de Bancos de Dados Públicos

Directórios

Planilha Query Builder

```

SELECT
    evt.EONET_ID,
    -- Aplicamos MAX() ao título. Como só há um título por evento, o resultado é o mesmo.
    MAX(JSON_VALUE(evt.JSON, '$.title')) AS titulo_do_evento,
    COUNT(ee.TB_ENDEREÇO3_ID_ENDEREÇO) AS enderecos_afetados
FROM
    TB_EONET3 evt
JOIN
    TB_ENDERECOEVENTOS3 ee ON evt.ID_EONET = ee.TB_EONET3_ID_EONET
GROUP BY
    -- Agrupamos apenas pelo identificador único do evento.
    evt.ID_EONET,
    evt.EONET_ID
HAVING
    COUNT(ee.TB_ENDEREÇO3_ID_ENDEREÇO) > 0
ORDER BY
    enderecos_afetados DESC;
  
```

Salida do Script x Resultado da Consulta x

SQL | Todas as Linhas Extraídas: 0 em 0,043 segundos

EONET_ID | TITULO_D... | ENDEREC...

